ArrayList Study Guide - Java

1. Introduction to ArrayList

What is ArrayList?

ArrayList is a resizable array implementation in Java that belongs to the `java.util` package. Unlike regular arrays, ArrayLists can grow and shrink dynamically during runtime.

Key Features of ArrayList

- **Dynamic Size**: Can resize automatically when elements are added or removed
- **Indexed Access**: Elements can be accessed using index positions (0-based)
- **Allows Duplicates**: Can store duplicate values
- **Ordered Collection**: Maintains insertion order
- **Generic Support**: Can specify the type of elements it stores

Importing ArrayList

```java
import java.util.ArrayList;
```

Creating an ArrayList

```java
// Creating an ArrayList of Strings
ArrayList<String> names = new ArrayList<String>();

// Creating an ArrayList of Integers
ArrayList<Integer> numbers = new ArrayList<Integer>();

// Using diamond operator (Java 7+)
ArrayList<String> cities = new ArrayList<>();
```

Common ArrayList Methods

Adding Elements

```java
ArrayList<String> fruits = new ArrayList<>();
fruits.add("Apple");        // Adds to the end
fruits.add("Banana");
fruits.add(1, "Orange");    // Adds at specific index
```

Accessing Elements

```java
String firstFruit = fruits.get(0);    // Gets element at index 0
int size = fruits.size();             // Gets the size of ArrayList
```

Modifying Elements

```java
fruits.set(0, "Mango");              // Replaces element at index 0
```

Removing Elements

```java
fruits.remove(0);                    // Removes element at index 0
fruits.remove("Banana");             // Removes specific element
fruits.clear();                      // Removes all elements
```

Checking Elements

```java
boolean hasApple = fruits.contains("Apple");    // Checks if element exists
boolean isEmpty = fruits.isEmpty();             // Checks if ArrayList is empty
int index = fruits.indexOf("Orange");           // Gets index of element
```

Example: Complete ArrayList Program

```java
import java.util.ArrayList;

public class ArrayListExample {
    public static void main(String[] args) {
        // Create ArrayList
        ArrayList<String> students = new ArrayList<>();

        // Add elements
        students.add("Alice");
        students.add("Bob");
        students.add("Charlie");

        // Display elements
        System.out.println("Students: " + students);

        // Access specific element
        System.out.println("First student: " + students.get(0));

        // Modify element
        students.set(1, "Robert");

        // Check size
        System.out.println("Number of students: " + students.size());

        // Remove element
        students.remove("Charlie");

        // Display final list
        System.out.println("Final list: " + students);
    }
}
```

2. Memory Management of ArrayList

Internal Structure

ArrayList internally uses a regular array to store elements. When the array becomes full, ArrayList creates a new larger array and copies all elements to it.

Initial Capacity

- Default initial capacity is **10 elements**

- You can specify initial capacity: `ArrayList<String> list = new ArrayList<>(20);`

Dynamic Resizing Process

1. **Growth**: When the internal array is full, ArrayList creates a new array with **1.5 times** the current capacity
2. **Copy**: All existing elements are copied to the new array
3. **Reference Update**: The internal reference points to the new array
4. **Garbage Collection**: The old array becomes eligible for garbage collection

Memory Growth Example

```java
// Initial capacity: 10
ArrayList<Integer> numbers = new ArrayList<>();

// After adding 11th element:
// New capacity = 10 * 1.5 = 15

// After adding 16th element:
// New capacity = 15 * 1.5 = 22 (approximately)
```

Memory Management Best Practices

1. Specify Initial Capacity

```java
// Good: If you know you'll store 100 elements
ArrayList<String> list = new ArrayList<>(100);

// Avoid: Using default capacity for large datasets
ArrayList<String> list = new ArrayList<>(); // Default: 10
```

2. Use trimToSize() Method

```java
ArrayList<String> list = new ArrayList<>(1000);
// Add only 50 elements
// Trim unused capacity
list.trimToSize(); // Reduces capacity to actual size
```

3. Consider Memory vs Performance Trade-offs

- **Frequent Resizing**: Impacts performance due to array copying
- **Large Initial Capacity**: Uses more memory but avoids resizing
- **trimToSize()**: Saves memory but may require resizing if elements are added later

Memory Visualization

```
Initial ArrayList (capacity: 10):
[element1][element2][    ][    ][    ][    ][    ][    ][    ][    ]

After adding 11th element (capacity: 15):
[element1][element2]...[element11][    ][    ][    ][    ]
```

ArrayList vs Array Memory Comparison

```java
// Regular Array - Fixed size, less memory overhead
String[] array = new String[10];

// ArrayList - Dynamic size, more memory overhead
ArrayList<String> list = new ArrayList<>();
```

3. Practice Questions

Basic ArrayList Operations

1. Create an ArrayList of integers and add the numbers 10, 20, 30, 40, 50
2. Remove the element at index 2 and display the remaining elements
3. Check if the ArrayList contains the number 30
4. Replace the first element with 100

Intermediate Operations

5. Create an ArrayList of strings representing colors. Add at least 5 colors, then remove all colors that start with the letter "B"
6. Write a method that takes an ArrayList of integers and returns the sum of all elements
7. Create two ArrayLists of integers and merge them into a third ArrayList

Advanced Operations

8. Write a program that removes duplicate elements from an ArrayList of strings
9. Create an ArrayList of integers, sort it in descending order without using built-in sort methods
10. Write a method that finds the second largest element in an ArrayList of integers

---

Quiz Questions

Question 1: What is the default initial capacity of an ArrayList in Java? a) 5
b) 10
c) 16
d) 20

Answer: b) 10

Question 2: Which method is used to add an element at a specific index in an ArrayList? a) insert(index, element)
b) add(index, element)
c) set(index, element)
d) put(index, element)

Answer: b) add(index, element)

Question 3: When an ArrayList reaches its capacity and needs to grow, by what factor does it typically increase its size? a) 2 times (doubles)
b) 1.5 times
c) 3 times
d) Adds 10 more elements

Answer: b) 1.5 times

Question 4: Which method would you use to check if an ArrayList is empty? a) size() == 0
b) isEmpty()
c) length() == 0
d) Both a and b

Answer: d) Both a and b

Question 5: What happens when you try to access an element at an invalid index in an ArrayList? a) Returns null
b) Returns the last element
c) Throws IndexOutOfBoundsException
d) Returns an empty string

Answer: c) Throws IndexOutOfBoundsException

Practice Coding Questions

Question 1: Write a Java program that creates an ArrayList of student names, allows the user to add 5 names, and then displays all names in reverse order.

Question 2: Create a method called `findMaximum` that takes an ArrayList of integers as a parameter and returns the largest number in the list.

Question 3: Write a program that creates two ArrayLists: one for even numbers (2, 4, 6, 8, 10) and one for odd numbers (1, 3, 5, 7, 9). Combine both lists into a third ArrayList and display the result.

Question 4: Create a method that takes an ArrayList of strings and returns a new ArrayList containing only the strings that have more than 5 characters.

Question 5: Write a program that creates an ArrayList of integers from 1 to 20, then removes all even numbers and displays the remaining odd numbers.